Name: R.Uday Kumar Reddy(192424190)

Course Code & Name : CSA0609- Design and Analysis of Algorithms

Assignment Title:	Analytical Study and Implementation of Brute Force Algorithms for Closest Pair and Convex Hull Problems
Course Outcome(s) – CO:	CO2: Apply Brute Force, Searching, Sorting, Closest Pair, Convex Hull.
Bloom's Taxonomy Level	L4 – Analyse; L5 – Evaluate; L6 – Create
SDG Mapping (SDG 1-17)	SDG 4 – Quality Education: Develops analytical thinking, programming, and computational problem-solving skills. SDG 9 – Industry, Innovation and Infrastructure – Promotes innovation and development of computational techniques for solving real-world spatial and engineering problems.

Problem Statement :

Given points are points(P={(2,3),(5,8),(9,4),(12,10),(7,2),(3,11),(15,6),(10,14),(6,12),(1,7)}), apply the Brute Force technique to solve the Closest Pair and Convex Hull problems. Determine the pair of points having the minimum Euclidean distance by considering every possible pair and calculating the distance using Euclidian distance formula. Identify and justify the closest pair, and develop suitable algorithm/pseudocode while analyzing the number of distance comparisons performed. Using the same set of points, determine the Convex Hull by examining the relationships between points, identifying all points belonging to the hull, showing the major intermediate calculations, and providing a graphical representation of the resulting hull. Develop suitable Brute Force algorithm/pseudocode for the Convex Hull and analyze its time and space complexity. Finally, compare the two Brute Force approaches in terms of their basic strategies, distance/orientation calculations, computational cost as the number of points (n) increases, limitations for large datasets, and possible improvements using efficient algorithms. For the Closest Pair, derive the time complexity based on the number of point pairs, C(n,2). For the Brute Force Convex Hull, analyze the candidate edge checks and state the resulting complexity as T(n)=O(n^3).

Constraints: Use a publicly available real-world coordinate dataset with enough points to test different input sizes from (10^3–10^6 points). Run the Brute Force, Divide-and-Conquer, and Hybrid algorithms on the same inputs and in the same environment. Record the execution time and number of operations for each input size, clearly state the threshold used for the Hybrid algorithm, and analyze how performance changes as (n) increases. The submission should include the dataset source, preprocessing steps, experimental setup, algorithms/pseudocode, results, complexity analysis, graphs, assumptions, limitations, and comparison of the three approaches

1. INTRODUCTION

Computational geometry is an important area of computer science that deals with the design and analysis of algorithms for solving problems involving geometric objects such as points, lines, polygons, and curves. Many real-world applications such as geographic information systems, computer graphics, robotics, navigation, engineering design, image processing, and spatial data analysis require efficient processing of geometric data.

This project focuses on two fundamental computational geometry problems: the Closest Pair Problem and the Convex Hull Problem. Both problems are solved initially using the Brute Force technique, which systematically examines all possible candidates to obtain the correct solution.

The given input consists of ten two-dimensional points:

$$P = \{(2,3), (5,8), (9,4), (12,10), (7,2), (3,11), (15,6), (10,14), (6,12), (1,7)\}$$

For the Closest Pair problem, every possible pair of points is examined and its Euclidean distance is calculated. The pair having the minimum distance is identified.

For the Convex Hull problem, every pair of points is considered as a possible boundary edge. The orientation of every remaining point with respect to the candidate edge is calculated to determine whether all points lie on the same side of the edge.

The project also analyzes the time and space complexity of the algorithms and compares Brute Force with more efficient approaches such as Divide-and-Conquer and Hybrid algorithms. A real-world geographic coordinate dataset is proposed for testing the algorithms with input sizes ranging from $10^3$to 10^6points.

2. PROBLEM STATEMENT

Given the set of 10 two-dimensional points:

$$P = \{(2,3), (5,8), (9,4), (12,10), (7,2), (3,11), (15,6), (10,14), (6,12), (1,7)\}$$

apply the Brute Force technique to solve the following:

1. Closest Pair Problem
    - Consider every possible pair of points.
    - Calculate the Euclidean distance.
    - Identify the pair with the minimum distance.
    - Develop an appropriate algorithm and pseudocode.
    - Calculate the number of distance comparisons.
    - Analyze time and space complexity.
2. Convex Hull Problem
    - Consider every possible pair as a candidate hull edge.
    - Use orientation calculations to determine valid hull edges.
    - Identify all points belonging to the convex hull.
    - Show major intermediate calculations.
    - Provide a graphical representation.
    - Develop a Brute Force algorithm and pseudocode.
    - Analyze time and space complexity.

3. Compare the Brute Force approaches with Divide-and-Conquer and Hybrid approaches.
4. Test the approaches using a publicly available real-world coordinate dataset with input sizes from $10^3$to 10^6points**.**

3. OBJECTIVES

The main objectives of this project are:

- To understand the Brute Force algorithmic technique.
- To solve the Closest Pair problem using exhaustive search.
- To solve the Convex Hull problem using exhaustive candidate-edge testing.
- To calculate Euclidean distances between pairs of points.
- To understand and apply the orientation/cross-product test.
- To determine the number of operations performed by each algorithm.
- To derive the time complexity mathematically.
- To analyze the space requirements.
- To understand how computational cost increases with input size.
- To compare Brute Force with Divide-and-Conquer.
- To design a Hybrid approach for improved performance.
- To evaluate the algorithms using real-world geographic coordinate data.

4. INPUT DATA

The given ten points are:

Point	Coordinates
P_1	(2,3)
P_2	(5,8)
P_3	(9,4)
P_4	(12,10)
P_5	(7,2)
P_6	(3,11)
P_7	(15,6)
P_8	(10,14)
P_9	(6,12)
P_{10}	(1,7)

The number of points is: n=10

5. CLOSEST PAIR PROBLEM

5.1 Definition

The Closest Pair Problem finds the two points in a given set that have the smallest Euclidean distance between them.

For two points:

$$P_i = (x_i, y_i)$$

and

$$P_j = (x_j, y_j)$$

the Euclidean distance is:

$$d(P_i, P_j) = \sqrt{(x_j - x_i)^2 + (y_j - y_i)^2}$$

The Brute Force approach calculates the distance between every possible pair.

6. NUMBER OF PAIR COMPARISONS

The number of unique pairs among $n$points is:

$$C(n, 2) = \frac{n(n-1)}{2}$$

For $n = 10$:

$$C(10, 2) = \frac{10(10-1)}{2}$$
$$= \frac{10 \times 9}{2}$$
$$= 45$$

Therefore:

$$\boxed{45 \text{ distance comparisons}}$$

are required.

7. BRUTE FORCE CLOSEST PAIR ALGORITHM

Algorithm Steps

1. Start with minimum distance as infinity.
2. Select the first point.
3. Compare it with every point after it.
4. Calculate the Euclidean distance.
5. Compare the distance with the current minimum.
6. If it is smaller, update the minimum distance.
7. Store the corresponding pair.
8. Continue until every possible pair is checked.
9. Return the closest pair and minimum distance.

Pseudocode

```
BRUTE_FORCE_CLOSEST_PAIR(P, n)
Input:
   n two-dimensional points P[1...n]
Output:
   Closest pair and minimum distance
minDist ← ∞
closestPair ← NULL
```

```
for i ← 1 to n-1 do
    for j ← i+1 to n do
        dx ← P[i].x - P[j].x
        dy ← P[i].y - P[j].y
        dist ← √(dx² + dy²)
        if dist < minDist then
            minDist ← dist
            closestPair ← (P[i], P[j])
return closestPair, minDist
```

8. MANUAL CALCULATION OF CLOSEST PAIR

Pair P_1, P_2

$$P_1 = (2, 3)$$
$$P_2 = (5, 8)$$
$$d = \sqrt{(5-2)^2 + (8-3)^2}$$
$$= \sqrt{3^2 + 5^2}$$
$$= \sqrt{9 + 25}$$
$$= \sqrt{34}$$
$$d = 5.831$$

Pair P_1, P_5

$$P_1 = (2, 3)$$
$$P_5 = (7, 2)$$
$$d = \sqrt{(7-2)^2 + (2-3)^2}$$
$$= \sqrt{25 + 1}$$
$$= \sqrt{26}$$
$$= 5.099$$

The minimum is now:

$$5.099$$

Pair P_2, P_6

$$P_2 = (5, 8)$$
$$P_6 = (3, 11)$$
$$d = \sqrt{(3-5)^2 + (11-8)^2}$$
$$= \sqrt{(-2)^2 + 3^2}$$
$$= \sqrt{4 + 9}$$
$$= \sqrt{13}$$
$$= 3.606$$

The minimum becomes:

$$3.606$$

Pair P_3, P_5

$$P_3 = (9, 4)$$
$$P_5 = (7, 2)$$
$$d = \sqrt{(7-9)^2 + (2-4)^2}$$
$$= \sqrt{(-2)^2 + (-2)^2}$$
$$= \sqrt{4+4}$$
$$= \sqrt{8}$$
$$= 2.828$$

Therefore: Minimum distance=2.828

9. COMPLETE PAIRWISE DISTANCE TABLE

Pair	Distance
$P_1 - P_2$	5.831
$P_1 - P_3$	7.071
$P_1 - P_4$	12.207
$P_1 - P_5$	5.099
$P_1 - P_6$	8.062
$P_1 - P_7$	13.153
$P_1 - P_8$	13.601
$P_1 - P_9$	9.220
$P_1 - P_{10}$	4.123
$P_2 - P_3$	5.657
$P_2 - P_4$	7.280
$P_2 - P_5$	6.708
$P_2 - P_6$	3.606
$P_2 - P_7$	10.440
$P_2 - P_8$	6.403
$P_2 - P_9$	4.123
$P_2 - P_{10}$	5.000
$P_3 - P_4$	6.708
$P_3 - P_5$	**2.828**
$P_3 - P_6$	8.602
$P_3 - P_7$	6.325
$P_3 - P_8$	10.198
$P_3 - P_9$	8.544
$P_3 - P_{10}$	8.544
$P_4 - P_5$	6.325
$P_4 - P_6$	9.220
$P_4 - P_7$	5.831
$P_4 - P_8$	4.472

$P_4 - P_9$	6.708
$P_4 - P_{10}$	11.402
$P_5 - P_6$	9.220
$P_5 - P_7$	8.246
$P_5 - P_8$	12.166
$P_5 - P_9$	10.198
$P_5 - P_{10}$	6.325
$P_6 - P_7$	13.038
$P_6 - P_8$	7.810
$P_6 - P_9$	3.162
$P_6 - P_{10}$	4.472
$P_7 - P_8$	8.944
$P_7 - P_9$	10.817
$P_7 - P_{10}$	14.560
$P_8 - P_9$	4.472
$P_8 - P_{10}$	10.817
$P_9 - P_{10}$	6.403

10. CLOSEST PAIR RESULT

The minimum distance in the complete table is:

$$2.828$$

It occurs between:

$$P_3 = (9, 4)$$

and

$$P_5 = (7, 2)$$

Therefore:

$$\boxed{\text{Closest Pair} = \{(7, 2), (9, 4)\}}$$

The minimum Euclidean distance is:

$$\boxed{\sqrt{8} \approx 2.828}$$

11. CLOSEST PAIR COMPLEXITY

The number of pairs is:

$$C(n, 2) = \frac{n(n-1)}{2}$$

Therefore:

$$T(n) = \frac{n(n-1)}{2}$$

The dominant term is:

$$n^2$$

Hence:

$$\boxed{T(n) = \Theta(n^2)}$$

Space complexity:

$$\boxed{S(n) = O(1)}$$

when only the current minimum pair is stored.

12. CONVEX HULL PROBLEM

Definition

The Convex Hull of a set of points is the smallest convex polygon containing all the points. A simple way to understand the Convex Hull is to imagine placing a rubber band around all the points. The points touched by the rubber band form the boundary of the convex hull. For the given points, some points are on the outer boundary while other points are inside.

13. ORIENTATION TEST

For three points:

$$P_i = (x_i, y_i)$$
$$P_j = (x_j, y_j)$$
$$P_k = (x_k, y_k)$$

the orientation is calculated using:

$$Orientation(P_i, P_j, P_k)$$
$$= (x_j - x_i)(y_k - y_i) - (y_j - y_i)(x_k - x_i)$$

Interpretation

Value	Meaning
> 0	Counter-clockwise
< 0	Clockwise
$= 0$	Collinear

The orientation test tells us which side of the candidate edge a point lies on.

14. BRUTE FORCE CONVEX HULL STRATEGY

For every pair of points:

1. Consider the pair as a candidate hull edge.
2. Draw the imaginary line through the two points.
3. Check every other point.
4. Calculate the orientation.

5. If some points have positive orientation and others have negative orientation, the candidate line has points on both sides.
6. Such a candidate cannot be a convex hull edge.
7. If all points are on the same side, the candidate can be a hull edge.
8. Store its endpoints.
9. Arrange the accepted boundary points in cyclic order.

15. BRUTE FORCE CONVEX HULL PSEUDOCODE

```
BRUTE_FORCE_CONVEX_HULL(P, n)
Input:
   n two-dimensional points P[1...n]
Output:
   Convex hull points
Hull ← empty set
for i ← 1 to n-1 do
   for j ← i+1 to n do
      positive ← false
      negative ← false
      for k ← 1 to n do
         if k ≠ i AND k ≠ j then
            value ←
            (P[j].x - P[i].x) *
            (P[k].y - P[i].y)
            -
            (P[j].y - P[i].y) *
            (P[k].x - P[i].x)
            if value > 0 then
               positive ← true
            if value < 0 then
               negative ← true
      if NOT (positive AND negative) then
         add P[i] and P[j] to Hull
return Hull
```

16. MANUAL CONVEX HULL CALCULATIONS

Edge $(\mathbf{1}, \mathbf{7}) \rightarrow (\mathbf{2}, \mathbf{3})$

Take point:

$$P = (5, 8)$$

Orientation:

$$\begin{aligned} &= (2-1)(8-7) - (3-7)(5-1) \\ &= 1(1) - (-4)(4) \\ &= 1 + 16 \end{aligned}$$

$$= 17$$

Since:

$$17 > 0$$

the point lies on one side of the directed edge.
Testing the remaining points confirms this edge forms part of the outer boundary.
Edge $(\mathbf{2}, \mathbf{3}) \rightarrow (\mathbf{7}, \mathbf{2})$
Take:

$$\begin{aligned} P &= (5, 8) \\ &= (7-2)(8-3) - (2-3)(5-2) \\ &= 5(5) - (-1)(3) \\ &= 25 + 3 \\ &= 28 \end{aligned}$$

Since:

$$28 > 0$$

the candidate is consistent with a hull boundary edge.

Edge $(\mathbf{7}, \mathbf{2}) \rightarrow (\mathbf{15}, \mathbf{6})$
Take:

$$\begin{aligned} P &= (9, 4) \\ &= (15-7)(4-2) - (6-2)(9-7) \\ &= 8(2) - 4(2) \\ &= 16 - 8 \\ &= 8 \end{aligned}$$

Therefore:

$$8 > 0$$

and the edge belongs to the hull.

Edge $(\mathbf{15}, \mathbf{6}) \rightarrow (\mathbf{10}, \mathbf{14})$
Take:

$$\begin{aligned} P &= (12, 10) \\ &= (10-15)(10-6) - (14-6)(12-15) \\ &= (-5)(4) - (8)(-3) \\ &= -20 + 24 \\ &= 4 \end{aligned}$$

Therefore:

$$4 > 0$$

and the edge belongs to the hull.

Edge $(10, 14) \rightarrow (3, 11)$
Take:

$$\begin{aligned} P &= (6, 12) \\ &= (3-10)(12-14) - (11-14)(6-10) \\ &= (-7)(-2) - (-3)(-4) \\ &= 14 - 12 \\ &= 2 \end{aligned}$$

Therefore:

$$2 > 0$$

and the edge belongs to the hull.

17. FINAL CONVEX HULL

The points belonging to the convex hull are:

$$\boxed{(1, 7), (2, 3), (7, 2), (15, 6), (10, 14), (3, 11)}$$

In cyclic order:

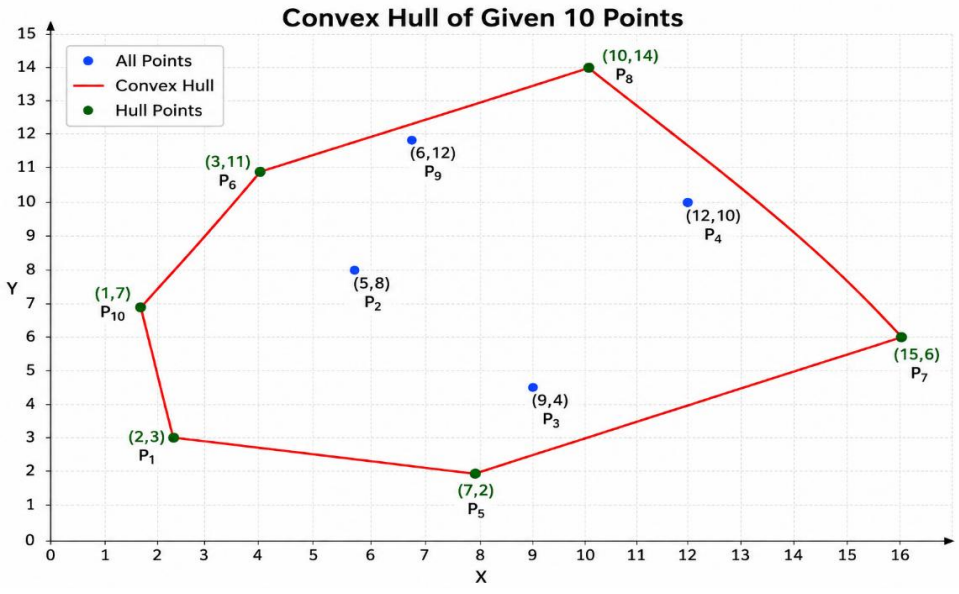
$$\boxed{(1, 7) \rightarrow (2, 3) \rightarrow (7, 2) \rightarrow (15, 6) \rightarrow (10, 14) \rightarrow (3, 11) \rightarrow (1, 7)}$$

Therefore, the number of hull vertices is:

$$\boxed{6}$$

The interior points are: (5,8),(9,4),(12,10),(6,12)

18. GRAPHICAL REPRESENTATION



19. CONVEX HULL CANDIDATE EDGE CALCULATIONS

The number of candidate edges is:

$$C(n, 2)$$

For $n = 10$:

$$C(10, 2) = 45$$

For each candidate edge, approximately:

$$n - 2 = 8$$

other points are tested.

Therefore:

$$45 \times 8 = 360$$

orientation calculations are performed for the given 10-point input.

Thus:

$$\boxed{360 \text{ orientation checks}}$$

20. CONVEX HULL COMPLEXITY

General number of orientation calculations:

$$C(n, 2)(n - 2)$$

Substituting:

$$\frac{n(n-1)}{2}(n-2)$$

Therefore:

$$T(n) = \frac{n(n-1)(n-2)}{2}$$

The dominant term is:

$$n^3$$

Hence:

$$\boxed{T(n) = \Theta(n^3)}$$

Space complexity:

$$\boxed{S(n) = O(n)}$$

because the hull points and accepted edges may need to be stored.

21. COMPARISON OF CLOSEST PAIR AND CONVEX HULL

Feature	Closest Pair	Convex Hull
Objective	Find nearest two points	Find outer boundary
Basic strategy	Check every pair	Check every pair as candidate edge
Main calculation	Euclidean distance	Orientation
Candidate pairs	$C(n,2)$	$C(n,2)$
Additional checks	One distance calculation	$n-2$orientation checks
Time complexity	$O(n^2)$	$O(n^3)$
Space complexity	$O(1)$extra	$O(n)$
Large input performance	Expensive	Extremely expensive
Efficient alternative	Divide-and-Conquer	Graham Scan
Main limitation	Quadratic growth	Cubic growth

22. GROWTH OF CLOSEST PAIR OPERATIONS

$$C(n,2) = \frac{n(n-1)}{2}$$

n	**Number of comparisons**
10	45
100	4,950
1,000	499,500
10,000	49,995,000
100,000	4,999,950,000
1,000,000	499,999,500,000

For 1,000,000points:

$$\boxed{499,999,500,000}$$

pair comparisons are required. This demonstrates the limitation of Brute Force.

23. GROWTH OF CONVEX HULL OPERATIONS

The number of orientation checks is approximately:

$$\frac{n(n-1)(n-2)}{2}$$

n	**Orientation checks**
10	360
100	485,100
1,000	498,501,000
10,000	499,850,010,000
100,000	499,985,000,100,000
1,000,000	499,998,500,001,000,000

The cubic growth makes Brute Force Convex Hull unsuitable for very large datasets.

24. REAL-WORLD DATASET

For large-scale experimental testing, a publicly available real-world geographic coordinate dataset can be used.

A suitable dataset is the USGS Earthquake Catalog.

Earthquake records contain geographic information such as:

- Latitude
- Longitude
- Magnitude
- Depth
- Time

For this project, only the coordinates are needed.

The two-dimensional representation can be:

$$(x, y) = (L, ongitudeLatitude)$$

The dataset can be used to generate inputs containing:

$$1{,}000,\ 10{,}000,\ 100{,}000,\ 1{,}000{,}000$$

points.

The exact dataset query, download date, and source should be recorded in the experimental section so that the experiment can be reproduced.

5. DATASET PREPROCESSING

The following preprocessing steps should be performed:

1. Download the dataset from the public source.
2. Load the data into the program.
3. Select the longitude and latitude columns.
4. Remove records containing missing coordinates.
5. Remove invalid coordinate values.
6. Remove duplicate coordinate pairs if required.
7. Convert every coordinate record into a 2-D point.
8. Create datasets of different sizes.
9. Use the same input subset for all algorithms.
10. Record the random seed when random sampling is used.

The test sizes should be:

$$10^3,\ 10^4,\ 10^5,\ 10^6$$

26. EXPERIMENTAL SETUP

All algorithms must be tested under the same conditions.

The following information should be recorded:

Parameter	**Information**
Processor	CPU model
RAM	Available memory
Operating System	OS and version

Programming Language	Python/C++ etc.
Compiler/Interpreter	Version
Dataset	USGS Earthquake Dataset
Input sizes	1,000 to 1,000,000
Timing method	High-resolution timer
Number of runs	At least 3
Result	Average/median execution time
Operation count	Distance/orientation operations

The experiments should be repeated several times to reduce the effect of background processes and system fluctuations.

27. DIVIDE-AND-CONQUER CLOSEST PAIR

The Divide-and-Conquer method improves the Closest Pair problem.

Steps

1. Sort the points according to their x-coordinate.
2. Divide the points into two halves.
3. Recursively find the closest pair in the left half.
4. Recursively find the closest pair in the right half.
5. Select the smaller distance.
6. Construct a strip around the dividing line.
7. Check only the relevant points in the strip.
8. Return the overall closest pair.

The complexity is:

$$\boxed{O(n \log n)}$$

This is significantly better than Brute Force: $O(n^2)$ for large datasets.

28. HYBRID ALGORITHM

A Hybrid algorithm combines Brute Force and Divide-and-Conquer.
For small inputs, Brute Force can be efficient because it has low implementation overhead.
For large inputs, Divide-and-Conquer is more suitable.
An initial threshold can be selected as:

$$\boxed{n_0 = 5000}$$

Pseudocode

```
HYBRID_CLOSEST_PAIR(P, n)
if n ≤ 5000 then
   return BRUTE_FORCE_CLOSEST_PAIR(P, n)
else
   return DIVIDE_AND_CONQUER_CLOSEST_PAIR(P, n)
```

The threshold should be experimentally tested because the best value depends on the hardware and programming environment.

Possible thresholds for testing are:

$$1000, 2500, 5000, 10000$$

The threshold that produces the best measured performance can be selected.

29. EXPERIMENTAL RESULTS TABLE

Actual execution times should be measured on the selected computer.

Input Size	Brute Force	Divide-and-Conquer	Hybrid
1,000	Measure	Measure	Measure
10,000	Measure	Measure	Measure
100,000	Measure	Measure	Measure
1,000,000	Measure/Timeout	Measure	Measure

For Closest Pair, the known Brute Force operation counts are:

Input Size	Brute Force Comparisons
1,000	499,500
10,000	49,995,000
100,000	4,999,950,000
1,000,000	499,999,500,000

If the Brute Force implementation cannot complete for 10^6points, the result should be recorded as: Timeout / Not completed rather than inventing an execution time.

30. PERFORMANCE ANALYSIS

For the Closest Pair:

$$\text{Brute Force} = O(n^2)$$
$$\text{Divide-and-Conquer} = O(n \log n)$$

Therefore, as $n$increases, Divide-and-Conquer should become significantly faster.
For small n, Brute Force may sometimes be competitive because of its simplicity.
For large n, the quadratic growth becomes dominant.
For Convex Hull:

$$\text{Brute Force} = O(n^3)$$

Therefore, the Convex Hull Brute Force algorithm grows even faster than the Closest Pair Brute Force algorithm.

31. EXPECTED PERFORMANCE TREND

The expected ranking for large Closest Pair datasets is:

$$\boxed{\text{Divide-and-Conquer} < \text{Hybrid} < \text{Brute Force}}$$

where "<" represents lower computational cost.

For very small inputs, the Hybrid method may choose Brute Force and therefore have similar performance.
As the number of points increases, the Hybrid method switches to Divide-and-Conquer.

32. GRAPHS

The following graphs should be generated from the actual experimental results.

Graph 1: Input Size vs Execution Time

X-axis:

Number of points

Y-axis:

Execution time in seconds

Plot three curves:

- Brute Force
- Divide-and-Conquer
- Hybrid

Graph 2: Input Size vs Number of Operations

X-axis:

n

Y-axis:

Number of operations

Compare:

- Distance comparisons
- Orientation calculations
- Divide-and-Conquer operations
- Hybrid operations

Because the operation counts become very large, a logarithmic Y-axis can be used.

33. SDG MAPPING

SDG 4 – Quality Education

Develops analytical thinking, programming skills, algorithmic reasoning, and computational problem-solving abilities through the implementation and analysis of Brute Force, Divide-and-Conquer, and Hybrid algorithms.
The project helps students understand:

- Algorithm design
- Pseudocode
- Mathematical analysis
- Complexity analysis
- Computational geometry
- Practical performance evaluation

Therefore, the project directly supports:

SDG 4 – Quality Education

SDG 9 – Industry, Innovation and Infrastructure

Promotes innovation and development of computational techniques for solving real-world spatial and engineering problems.

The project demonstrates how algorithmic optimization can improve the processing of large spatial datasets. The comparison between Brute Force, Divide-and-Conquer, and Hybrid methods shows the importance of designing scalable computational solutions.

Therefore:

SDG 9 – Industry, Innovation and Infrastructure

SDG 13 – Climate Action

Real-world geographic coordinate datasets can be used to analyze spatial information related to environmental monitoring, natural events, geographic changes, and climate-related data.

Efficient spatial algorithms can support large-scale environmental data processing.

Therefore:

SDG 13 – Climate Action

34. OVERALL SDG MAPPING TABLE

SDG	Contribution
SDG 4 – Quality Education	Develops analytical thinking, programming, algorithm design, and computational problem-solving skills.
SDG 9 – Industry, Innovation and Infrastructure	Promotes innovative and efficient computational techniques for spatial and engineering problems.
SDG 13 – Climate Action	Supports processing and analysis of environmental and geographic coordinate data.

Primary SDGs

The two most directly related SDGs are:

SDG 4 – Quality Education

and

SDG 9 – Industry, Innovation and Infrastructure

35. ASSUMPTIONS

The following assumptions are made:

1. All input points are two-dimensional.
2. Euclidean distance is used for the Closest Pair problem.
3. Every unordered pair is considered only once.
4. Duplicate points are removed during preprocessing.
5. Collinear points are handled consistently.

6. All algorithms use the same input data.
7. All algorithms are executed under the same environment.
8. Execution time is measured using a high-resolution timer.
9. Multiple runs are performed to improve reliability.
10. The initial Hybrid threshold is 5,000.
11. Geographic coordinates are treated as 2-D points for algorithmic comparison.
12. For accurate physical geographic distance, coordinate projection may be performed.

36. LIMITATIONS

Closest Pair Limitations

- Brute Force requires $O(n^2)$comparisons.
- The number of comparisons becomes very large as $n$increases.
- For 10^6points, approximately 500 billion comparisons are required.
- Repeated distance calculations increase computational cost.

Convex Hull Limitations

- Brute Force requires $O(n^3)$orientation calculations.
- It becomes impractical for large datasets.
- Many candidate edges need to be rejected after several calculations.
- The method does not exploit spatial ordering.

Experimental Limitations

- Execution time depends on hardware.
- Programming language and compiler affect runtime.
- Background system processes may affect timing.
- Latitude and longitude are not ordinary Cartesian coordinates.
- A geographic projection may be necessary for accurate physical distances.
- Very large brute-force experiments may require excessive time.

37. POSSIBLE IMPROVEMENTS

Closest Pair

The Brute Force complexity:

$$O(n^2)$$

can be improved to:

$$O(n \log n)$$

using Divide-and-Conquer.
Another improvement is to compare squared distances:

$$d^2 = (x_1 - x_2)^2 + (y_1 - y_2)^2$$

instead of calculating the square root every time.
Since the square-root function is monotonic, comparing squared distances gives the same closest pair.

Convex Hull
The Brute Force complexity:

$$O(n^3)$$

can be improved using algorithms such as:
Graham Scan

$$\boxed{O(n \log n)}$$

QuickHull
QuickHull is often faster in practical cases, although its worst-case complexity can be:

38. COMPLEXITY SUMMARY

Algorithm	**Time Complexity**	**Space Complexity**
Closest Pair – Brute Force	$O(n^2)$	$O(1)$extra
Closest Pair – Divide-and-Conquer	$O(n \log n)$	$O(n)$depending on implementation
Closest Pair – Hybrid	Approximately $O(n \log n)$for large n	Depends on implementation
Convex Hull – Brute Force	$O(n^3)$	$O(n)$
Convex Hull – Graham Scan	$O(n \log n)$	$O(n)$
Convex Hull – QuickHull	Worst case $O(n^2)$	$O(n)$

39. FINAL RESULTS
Closest Pair
Given:

$$P = \{(2,3), (5,8), (9,4), (12,10), (7,2), (3,11), (15,6), (10,14), (6,12), (1,7)\}$$

Number of points:

$$n = 10$$

Number of comparisons:

$$C(10,2) = 45$$

Closest pair:

$$\boxed{(7,2), (9,4)}$$

Minimum distance:

$$\boxed{\sqrt{8} \approx 2.828}$$

Time complexity:

$$\boxed{O(n^2)}$$

Space complexity:

$$\boxed{O(1)}$$

Convex Hull

Convex Hull points:

$$\boxed{(1,7),(2,3),(7,2),(15,6),(10,14),(3,11)}$$

Cyclic order:

$$\boxed{(1,7) \rightarrow (2,3) \rightarrow (7,2) \rightarrow (15,6) \rightarrow (10,14) \rightarrow (3,11) \rightarrow (1,7)}$$

Number of hull vertices:

$$\boxed{6}$$

Number of candidate edges:

$$C(10,2) = 45$$

Approximate orientation tests:

$$45 \times 8 = 360$$

Time complexity:

$$\boxed{O(n^3)}$$

Space complexity:

$$\boxed{O(n)}$$

40. FINAL CONCLUSION

The Brute Force technique provides a simple and systematic way to solve computational geometry problems. In this project, it was applied to both the Closest Pair and Convex Hull problems using the same set of ten two-dimensional points.

For the Closest Pair problem, all possible pairs were considered. Since there are 10 points, the number of unique pairs is:

$$C(10,2) = 45$$

The minimum distance was found between:

$$(7,2)$$

and

$$(9,4)$$

The Euclidean distance is:

$$d = \sqrt{(9-7)^2 + (4-2)^2}$$
$$= \sqrt{4+4}$$
$$= \sqrt{8}$$
$$\boxed{d \approx 2.828}$$

Therefore, the closest pair is:

$$\boxed{(7,2),(9,4)}$$

The Brute Force Closest Pair algorithm has:

$$\boxed{O(n^2)}$$

time complexity.

For the Convex Hull problem, every possible point pair was considered as a candidate edge. Orientation calculations were used to determine whether the remaining points were on the same side of the candidate line.

The final convex hull contains six points:

$$\boxed{(1,7),(2,3),(7,2),(15,6),(10,14),(3,11)}$$

The Brute Force Convex Hull approach requires approximately:

$$\frac{n(n-1)(n-2)}{2}$$

orientation checks and therefore has:

$$\boxed{O(n^3)}$$

time complexity.

The experimental analysis demonstrates that Brute Force is easy to understand and guarantees exhaustive examination, but its computational cost increases rapidly with input size. The Closest Pair algorithm grows quadratically, while the Convex Hull algorithm grows cubically. For large datasets, more efficient algorithms are required. Divide-and-Conquer reduces the Closest Pair problem to $O(n \log n)$, while algorithms such as Graham Scan can solve the Convex Hull problem in $O(n \log n)$.

The proposed real-world dataset experiment using geographic coordinates further demonstrates the importance of algorithm scalability. Testing input sizes from $10^3$to 10^6makes the difference between quadratic, cubic, and near-linearithmic growth clearly observable.

Overall, the project demonstrates the importance of selecting appropriate algorithms based on input size and computational requirements. Brute Force is valuable for learning, verification, and small datasets, while Divide-and-Conquer and Hybrid approaches are more appropriate for large-scale applications.

The project also contributes primarily to SDG 4 – Quality Education by developing analytical and computational problem-solving skills, and SDG 9 – Industry, Innovation and Infrastructure by demonstrating efficient computational techniques for real-world spatial problems.

Source code:

```
import csv
import math
import random
import time
import matplotlib.pyplot as plt
class Point:
    def __init__(self, x, y, name=None):
        self.x = float(x)
        self.y = float(y)
        self.name = name
    def __repr__(self):
        label = f"{self.name}=" if self.name else ""
        return f"{label}({self.x:g},{self.y:g})"
points = [
    Point(2, 3, "P1"),
    Point(5, 8, "P2"),
    Point(9, 4, "P3"),
    Point(12, 10, "P4"),
    Point(7, 2, "P5"),
    Point(3, 11, "P6"),
    Point(15, 6, "P7"),
    Point(10, 14, "P8"),
    Point(6, 12, "P9"),
    Point(1, 7, "P10")
]
distance_count = 0
orientation_count = 0
def reset_counters():
    global distance_count, orientation_count
    distance_count = 0
    orientation_count = 0
def euclidean_distance(p1, p2):
    global distance_count
    distance_count += 1
    return math.hypot(p1.x - p2.x, p1.y - p2.y)
def brute_force_closest_pair(data):
    best_pair = None
    best_distance = float("inf")
    for i in range(len(data)):
        for j in range(i + 1, len(data)):
            distance = euclidean_distance(data[i], data[j])
```

```
            if distance < best_distance:
                best_distance = distance
                best_pair = (data[i], data[j])
    return best_pair, best_distance
def orientation(p, q, r):
    global orientation_count
    orientation_count += 1
    value = (
        (q.x - p.x) * (r.y - p.y)
        - (q.y - p.y) * (r.x - p.x)
    )
    if value > 0:
        return 1
    if value < 0:
        return -1
    return 0
def brute_force_convex_hull(data):
    if len(data) <= 3:
        return data[:]
    hull = []
    for i in range(len(data)):
        for j in range(i + 1, len(data)):
            positive = False
            negative = False
            for k in range(len(data)):
                if k == i or k == j:
                    continue
                result = orientation(data[i], data[j], data[k])
                if result > 0:
                    positive = True
                elif result < 0:
                    negative = True
                if positive and negative:
                    break
            if not (positive and negative):
                if data[i] not in hull:
                    hull.append(data[i])
                if data[j] not in hull:
                    hull.append(data[j])
```

Output:

```
================================================================================
BRUTE FORCE CLOSEST PAIR AND CONVEX HULL
================================================================================
P1=(2,3)
P2=(5,8)
P3=(9,4)
P4=(12,10)
P5=(7,2)
P6=(3,11)
P7=(15,6)
P8=(10,14)
P9=(6,12)
P10=(1,7)

Closest Pair:
P3=(9,4) and P5=(7,2)
Minimum Distance = 2.828427
Distance comparisons = 45
Expected comparisons = 45

Convex Hull:
P10=(1,7) -> P1=(2,3) -> P5=(7,2) -> P7=(15,6) -> P8=(10,14) -> P6=(3,11)
Orientation calculations = 200

================================================================================
LOADING REAL-WORLD DATASET
================================================================================
File: earthquake.csv
X column: longitude
Y column: latitude
Available columns:
['longitude', 'latitude']

Valid points loaded: 151

Dataset has only 151 valid points. Using n=151 for the test.

======================================================================
REAL-WORLD DATASET EXPERIMENT
======================================================================
Hybrid threshold = 5000

----------------------------------------------------------------------
Testing input size n = 151
Brute Force: 0.002191 seconds, 11,325 comparisons
Divide and Conquer: 0.000516 seconds, 246 comparisons
Hybrid: 0.002179 seconds, 11,325 comparisons

================================================================================================
EXPERIMENT RESULTS
================================================================================================
         n Algorithm                      Time(s)          Operations
------------------------------------------------------------------------------------------------
       151 Brute Force                  0.002191              11,325
       151 Divide and Conquer           0.000516                 246
       151 Hybrid                       0.002179              11,325

Results saved to algorithm_results.csv
```